M02 — Prácticas integrales: programar y probar APIs REST con .NET 8

Autor: JAIME GALLO

Metodología de trabajo

Cada apartado parte del diseño REST explicado en teoría y culmina en **código ejecutable y peticiones HTTP**. Se trabaja con Visual Studio, ASP.NET Core 8 Minimal API, EF Core 8 y SQL Server Express LocalDB. Los cuatro estados de referencia están en `PROYECTO/2.1` a `PROYECTO/2.4`: no son simulaciones, sino proyectos con endpoints reales probados por HTTP.

La actividad práctica no requiere operaciones Git. El alumnado construye primero la etapa 2.1, incorpora los verbos HTTP en 2.2, conecta la validación en 2.3 y documenta el mismo servicio en 2.4. Para contrastar el resultado, puede abrir por separado los proyectos de referencia. La copia no sustituye la escritura y comprensión de código.

Persistencia: la API del alumno utiliza SQL Server Express LocalDB. Los tests de integración usan EF Core InMemory únicamente para ejecutarse de forma independiente en GitHub Actions. `EnsureCreated` se emplea en desarrollo para simplificar el arranque del ejercicio; no se propone para despliegues productivos.

Punto 2.1 — Del diseño REST a los primeros endpoints GET

Proyecto comprobado: `PROYECTO/2.1-diseno-apis-rest`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.1.1 — Crear el proyecto ASP.NET Core Empty desde Visual Studio

Objetivo del paso. Crear el proyecto ASP.NET Core Empty desde Visual Studio. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Archivo > Nuevo > Proyecto > ASP.NET Core Empty. Selecciona C#, .NET 8 y el nombre AceriaApp.Api. No marques autenticación. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. REST identifica órdenes mediante una URI estable. ASP.NET Core no pide implementar un validador de cadenas para disponer de enrutamiento: la llamada MapGet asocia un verbo y una plantilla de ruta al manejador real. Hay que distinguir la entidad Orden, que EF Core persiste, de la representación pública que recibe el consumidor. Conocer los atributos Id, Cliente, Material, Cantidad, Estado y FechaCreacion permite justificar después por qué no todos se aceptan como entrada.

Verificación concreta. Verifica el TargetFramework net8.0 y arranca el proyecto vacío. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.2 — Explorar Program.cs, Kestrel y el perfil de arranque

Objetivo del paso. Explorar Program.cs, Kestrel y el perfil de arranque. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Abre Program.cs y localiza CreateBuilder, Build y Run. Ejecuta con F5, observa la dirección del perfil local y detén Kestrel. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. REST identifica órdenes mediante una URI estable. ASP.NET Core no pide implementar un validador de cadenas para disponer de enrutamiento: la llamada MapGet asocia un verbo y una plantilla de ruta al manejador real. Hay que distinguir la entidad Orden, que EF Core persiste, de la representación pública que recibe el consumidor. Conocer los atributos Id, Cliente, Material, Cantidad, Estado y FechaCreacion permite justificar después por qué no todos se aceptan como entrada.

Verificación concreta. Kestrel escucha en el puerto anunciado; no es una prueba de base de datos. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.3 — Dibujar los recursos y decidir las dos URIs GET

Objetivo del paso. Dibujar los recursos y decidir las dos URIs GET. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Diseña las rutas GET /api/v1/ordenes y GET /api/v1/ordenes/{id:guid}. Explica qué recurso identifica cada una. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. REST identifica órdenes mediante una URI estable. ASP.NET Core no pide implementar un validador de cadenas para disponer de enrutamiento: la llamada MapGet asocia un verbo y una plantilla de ruta al manejador real. Hay que distinguir la entidad Orden, que EF Core persiste, de la representación pública que recibe el consumidor. Conocer los atributos Id, Cliente, Material, Cantidad, Estado y FechaCreacion permite justificar después por qué no todos se aceptan como entrada.

Verificación concreta. Las dos rutas usan sustantivos en plural, y el detalle tiene un Guid. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.4 — Crear carpeta Modelo y entidad Orden

Objetivo del paso. Crear carpeta Modelo y entidad Orden. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Añade la carpeta Modelo. Crea Orden.cs con Guid Id, Cliente, Material, Cantidad, Estado y FechaCreacion; compara con la clase de referencia. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: `src/AceriaApp.Api/Modelo/Orden.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
namespace AceriaApp.Api.Modelo;
// INICIO PASO 2.1.4
public sealed class Orden
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string Cliente { get; set; } = string.Empty;
    public string Material { get; set; } = string.Empty;
    public int Cantidad { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public DateTime FechaCreacion { get; set; } = DateTime.UtcNow;
}
// FIN PASO 2.1.4
```

Explicación conceptual. REST identifica órdenes mediante una URI estable. ASP.NET Core no pide implementar un validador de cadenas para disponer de enrutamiento: la llamada MapGet asocia un verbo y una plantilla de ruta al manejador real. Hay que distinguir la entidad Orden, que EF Core persiste, de la representación pública que recibe el consumidor. Conocer los atributos Id, Cliente, Material, Cantidad, Estado y FechaCreacion permite justificar después por qué no todos se aceptan como entrada.

Verificación concreta. La clase Orden compila, y el estado inicial es Pendiente. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.5 — Explicar ID Guid, Estado y FechaCreacion de la entidad

Objetivo del paso. Explicar ID Guid, Estado y FechaCreacion de la entidad. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En Orden.cs observa que Id se genera por defecto y FechaCreacion emplea UtcNow. Distingue los datos que proporciona el servidor de los del cliente. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. REST identifica órdenes mediante una URI estable. ASP.NET Core no pide implementar un validador de cadenas para disponer de enrutamiento: la llamada MapGet asocia un verbo y una plantilla de ruta al manejador real. Hay que distinguir la entidad Orden, que EF Core persiste, de la representación pública que recibe el consumidor. Conocer los atributos Id, Cliente, Material, Cantidad, Estado y FechaCreacion permite justificar después por qué no todos se aceptan como entrada.

Verificación concreta. No se reciben el Id ni la fecha desde una petición de creación. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.6 — Crear OrdenDto como contrato de salida

Objetivo del paso. Crear OrdenDto como contrato de salida. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En Modelo crea OrdenDto.cs como record público con los datos que va a serializar JSON; no lo conviertas en entidad de EF Core. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Modelo/OrdenDto.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
namespace AceriaApp.Api.Modelo;
// INICIO PASO 2.1.5
public sealed record OrdenDto(Guid Id, string Cliente, string Material,
    int Cantidad, string Estado, DateTime FechaCreacion)
{
    public static OrdenDto Desde(Orden x) =>
        new(x.Id,x.Cliente,x.Material,x.Cantidad,x.Estado,x.FechaCreacion);
}
// FIN PASO 2.1.5
```

Explicación conceptual. El record OrdenDto representa el JSON de salida. El método Desde transforma explícitamente la entidad a una representación. Esta distinción reduce el acoplamiento entre el esquema de la tabla y el contrato HTTP. Si se cambiara una propiedad interna de Orden no tendría por qué aparecer automáticamente en la respuesta; el método de conversión es precisamente el lugar donde decidir qué se publica.

Verificación concreta. El compilador distingue Orden de OrdenDto. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.7 — Programar la conversión OrdenDto.Desde

Objetivo del paso. Programar la conversión OrdenDto.Desde. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Añade a OrdenDto un método estático Desde(Orden x); construye el record usando las propiedades de una orden. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Modelo/OrdenDto.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
namespace AceriaApp.Api.Modelo;
// INICIO PASO 2.1.5
public sealed record OrdenDto(Guid Id, string Cliente, string Material,
    int Cantidad, string Estado, DateTime FechaCreacion)
{
    public static OrdenDto Desde(Orden x) =>
        new(x.Id,x.Cliente,x.Material,x.Cantidad,x.Estado,x.FechaCreacion);
}
// FIN PASO 2.1.5
```

Explicación conceptual. El record OrdenDto representa el JSON de salida. El método Desde transforma explícitamente la entidad a una representación. Esta distinción reduce el acoplamiento entre el esquema de la tabla y el contrato HTTP. Si se cambiara una propiedad interna de Orden no tendría por qué aparecer automáticamente en la respuesta; el método de conversión es precisamente el lugar donde decidir qué se publica.

Verificación concreta. El DTO contiene el mismo Id y datos públicos que la entidad. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.8 — Instalar EF Core 8 para SQL Server

Objetivo del paso. Instalar EF Core 8 para SQL Server. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Desde Administrar paquetes NuGet instala Microsoft.EntityFrameworkCore.SqlServer 8.x en AceriaApp.Api, o utiliza dotnet add package con versión 8. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. El record OrdenDto representa el JSON de salida. El método Desde transforma explícitamente la entidad a una representación. Esta distinción reduce el acoplamiento entre el esquema de la tabla y el contrato HTTP. Si se cambiara una propiedad interna de Orden no tendría por qué aparecer automáticamente en la respuesta; el método de conversión es precisamente el lugar donde decidir qué se publica.

Verificación concreta. La compilación reconoce UseSqlServer. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.9 — Crear AceriaDbContext y DbSet

Objetivo del paso. Crear AceriaDbContext y DbSet. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Crea Datos/AceriaDbContext.cs; haz que herede de DbContext y declara DbSet Ordenes. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Datos/AceriaDbContext.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Api.Datos;
// INICIO PASO 2.1.6
public sealed class AceriaDbContext(DbContextOptions<AceriaDbContext> options) :
DbContext(options)
{
```

```
    public DbSet<Orden> Ordenes => Set<Orden>();
    protected override void OnModelCreating(ModelBuilder b)
    {
        var e=b.Entity<Orden>();
        e.HasKey(x=>x.Id);
        e.Property(x=>x.Cliente).HasMaxLength(100).IsRequired();
        e.Property(x=>x.Material).HasMaxLength(100).IsRequired();
        e.Property(x=>x.Estado).HasMaxLength(24).IsRequired();
    }
}
// FIN PASO 2.1.6
```

Explicación conceptual. El record OrdenDto representa el JSON de salida. El método Desde transforma explícitamente la entidad a una representación. Esta distinción reduce el acoplamiento entre el esquema de la tabla y el contrato HTTP. Si se cambiara una propiedad interna de Orden no tendría por qué aparecer automáticamente en la respuesta; el método de conversión es precisamente el lugar donde decidir qué se publica.

Verificación concreta. Al compilar, el contexto conoce Orden y puede consultarla. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.10 — Configurar OnModelCreating, clave y longitudes

Objetivo del paso. Configurar OnModelCreating, clave y longitudes. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En OnModelCreating configura HasKey(Id), las longitudes de Cliente y Material y los campos obligatorios. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. El record OrdenDto representa el JSON de salida. El método Desde transforma explícitamente la entidad a una representación. Esta distinción reduce el acoplamiento entre el esquema de la tabla y el contrato HTTP. Si se cambiara una propiedad interna de Orden no tendría por qué aparecer automáticamente en la respuesta; el método de conversión es precisamente el lugar donde decidir qué se publica.

Verificación concreta. El modelo restringe tamaños de columna y reconoce la clave. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.11 — Configurar la cadena de conexión a LocalDB

Objetivo del paso. Configurar la cadena de conexión a LocalDB. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En appsettings.json añade ConnectionStrings:AceriaApp con (localdb)\MSSQLLocalDB y Database=AceriaAppM02. No sustituyas LocalDB por SQLite. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. El contexto contiene DbSet y las restricciones de las columnas. La API se ejecuta sobre SQL Server Express LocalDB en el ordenador del alumno. Registrar AddDbContext permite que los endpoints reciban el contexto mediante inyección de dependencias: no hay que crear el contexto con new dentro de cada manejador. En tests se sustituye solamente el proveedor por InMemory para evitar depender de Windows.

Verificación concreta. La cadena se lee desde Configuration y apunta al servidor local. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.12 — Registrar AceriaDbContext mediante AddDbContext

Objetivo del paso. Registrar AceriaDbContext mediante AddDbContext. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En Program.cs, antes de builder.Build, añade AddDbContext con UseSqlServer y GetConnectionString. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.1.7: SQL Server Express LocalDB, no SQLite.
builder.Services.AddDbContext<AceriaDbContext>(o=>
    o.UseSqlServer(builder.Configuration.GetConnectionString("AceriaApp")));

var app=builder.Build();
// EnsureCreated se utiliza SOLO como ayuda didáctica en Development.
// Para un sistema productivo se utilizarían migraciones controladas.
if(app.Environment.IsDevelopment()&&!
app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope=app.Services.CreateScope();
    await
scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreatedAsync();
}
```

Explicación conceptual. El contexto contiene DbSet y las restricciones de las columnas. La API se ejecuta sobre SQL Server Express LocalDB en el ordenador del alumno. Registrar AddDbContext permite que los endpoints reciban el contexto mediante inyección de dependencias: no hay que crear el contexto con new dentro de cada manejador. En tests se sustituye solamente el proveedor por InMemory para evitar depender de Windows.

Verificación concreta. El manejador HTTP podrá recibir AceriaDbContext por inyección. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.13 — Crear la base didáctica solo en Development

Objetivo del paso. Crear la base didáctica solo en Development. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En Development crea la base de enseñanza con Database.EnsureCreatedAsync usando un scope; deja escrito que en producción se usarían migraciones. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. El contexto contiene DbSet y las restricciones de las columnas. La API se ejecuta sobre SQL Server Express LocalDB en el ordenador del alumno. Registrar AddDbContext permite que los endpoints reciban el contexto mediante inyección de dependencias: no hay que crear el contexto con new dentro de cada manejador. En tests se sustituye solamente el proveedor por InMemory para evitar depender de Windows.

Verificación concreta. Al arrancar en Windows aparece la base AceriaAppM02 si LocalDB está instalado. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.14 — Escribir el endpoint GET de colección

Objetivo del paso. Escribir el endpoint GET de colección. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En Program.cs escribe MapGet de /api/v1/ordenes y recibe AceriaDbContext como argumento del lambda. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.1.8: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")

;
```

Explicación conceptual. El contexto contiene DbSet y las restricciones de las columnas. La API se ejecuta sobre SQL Server Express LocalDB en el ordenador del alumno. Registrar AddDbContext permite que los endpoints reciban el contexto mediante inyección de dependencias: no hay que crear el contexto con new dentro de cada manejador. En tests se sustituye solamente el proveedor por InMemory para evitar depender de Windows.

Verificación concreta. El endpoint queda registrado y responde a GET. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.15 — Utilizar AsNoTracking y proyectar a DTO

Objetivo del paso. Utilizar AsNoTracking y proyectar a DTO. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Dentro de MapGet consulta db.Ordenes.AsNoTracking, materializa la lista y aplica OrdenDto.Desde a cada entidad. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. El contexto contiene DbSet y las restricciones de las columnas. La API se ejecuta sobre SQL Server Express LocalDB en el ordenador del alumno. Registrar AddDbContext permite que los endpoints reciban el contexto mediante inyección de dependencias: no hay que crear el contexto con new dentro de cada manejador. En tests se sustituye solamente el proveedor por InMemory para evitar depender de Windows.

Verificación concreta. La salida expone DTOs y no instancias de la entidad EF Core. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.16 — Ejecutar GET y verificar 200 con lista vacía

Objetivo del paso. Ejecutar GET y verificar 200 con lista vacía. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Abre AceriaApp.http, ejecuta GET de colección y comprueba el código y el JSON aunque la tabla esté vacía. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. El manejador obtiene AceriaDbContext, usa AsNoTracking para consultar sin seguimiento y devuelve Results.Ok con una secuencia de DTOs. En una colección vacía se responde 200 y una lista JSON vacía, pues la colección como recurso existe aunque no contenga elementos. La respuesta debe inspeccionarse desde AceriaApp.http, no deducirse de que el código compile.

Verificación concreta. La respuesta es 200 y un arreglo JSON; no 404. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.17 — Escribir el GET individual con

Objetivo del paso. Escribir el GET individual con {id:guid}. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Añade MapGet /api/v1/ordenes/{id:guid} y recibe Guid id y DbContext inyectado. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.1.9: recurso individual.
app.MapGet("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(x=>x.Id==id);
    IResult response=orden is null ? Results.NotFound() : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")

;

app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Explicación conceptual. El manejador obtiene AceriaDbContext, usa AsNoTracking para consultar sin seguimiento y devuelve Results.Ok con una secuencia de DTOs. En una colección vacía se responde 200 y una lista JSON vacía, pues la colección como recurso existe aunque no contenga elementos. La respuesta debe inspeccionarse desde AceriaApp.http, no deducirse de que el código compile.

Verificación concreta. La ruta coincide con un identificador Guid válido. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.18 — Devolver NotFound cuando no existe la orden

Objetivo del paso. Devolver NotFound cuando no existe la orden. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Busca mediante FirstOrDefaultAsync y devuelve Results.NotFound cuando la orden no existe; usa Results.Ok cuando aparece. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. El manejador obtiene AceriaDbContext, usa AsNoTracking para consultar sin seguimiento y devuelve Results.Ok con una secuencia de DTOs. En una colección vacía se responde 200 y una lista JSON vacía, pues la colección como recurso existe aunque no contenga elementos. La respuesta debe inspeccionarse desde AceriaApp.http, no deducirse de que el código compile.

Verificación concreta. Un GUID nuevo devuelve 404 sin excepción 500. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.19 — Comprobar diferencias entre ruta incorrecta y recurso ausente

Objetivo del paso. Comprobar diferencias entre ruta incorrecta y recurso ausente. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Envía una ruta con identificador que no es GUID y otra con GUID inexistente; anota qué parte resuelve el enrutador y cuál el manejador. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. El manejador obtiene AceriaDbContext, usa AsNoTracking para consultar sin seguimiento y devuelve Results.Ok con una secuencia de DTOs. En una colección vacía se responde 200 y una lista JSON vacía, pues la colección como recurso existe aunque no contenga elementos. La respuesta debe inspeccionarse desde AceriaApp.http, no deducirse de que el código compile.

Verificación concreta. Comprendes la diferencia entre plantilla que no coincide y recurso ausente. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.20 — Crear AceriaApp.http y enviar peticiones desde Visual Studio

Objetivo del paso. Crear AceriaApp.http y enviar peticiones desde Visual Studio. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En Visual Studio abre o crea AceriaApp.http, define @Host y dos peticiones GET con la ruta versionada. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. El manejador obtiene AceriaDbContext, usa AsNoTracking para consultar sin seguimiento y devuelve Results.Ok con una secuencia de DTOs. En una colección vacía se responde 200 y una lista JSON vacía, pues la colección como recurso existe aunque no contenga elementos. La respuesta debe inspeccionarse desde AceriaApp.http, no deducirse de que el código compile.

Verificación concreta. Puedes ejecutar ambos GET sin copiar URLs del navegador. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.21 — Crear el proyecto xUnit de pruebas HTTP

Objetivo del paso. Crear el proyecto xUnit de pruebas HTTP. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Crea un proyecto xUnit AceriaApp.Api.Tests y referencia AceriaApp.Api; instala Microsoft.AspNetCore.Mvc.Testing 8. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`tests/AceriaApp.Api.Tests/ApiFactory.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Datos;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.AspNetCore.TestHost;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
```

```
using Microsoft.Extensions.DependencyInjection.Extensions;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.1.12: HTTP real; base en memoria SOLO en tests automatizados.
public sealed class ApiFactory:WebApplicationFactory<Program>
{
    private readonly string _databaseName="Tests-"+Guid.NewGuid();
    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Development");
        builder.UseSetting("SkipDatabaseInitialization","true");
        builder.ConfigureTestServices(services=>
        {
            services.RemoveAll<DbContextOptions<AceriaDbContext>>();
            services.AddDbContext<AceriaDbContext>(opt=>
                opt.UseInMemoryDatabase(_databaseName));
        });
    }
}
```

Explicación conceptual. La plantilla {id:guid} restringe el parámetro de ruta a GUID válidos. El manejador consulta una orden específica y decide entre Results.Ok y Results.NotFound. Una petición a un GUID inexistente no debería convertirse en 500 ni en una respuesta 200 con contenido vacío. WebApplicationFactory inicia la aplicación en memoria y permite comprobar 200 y 404 sobre los endpoints auténticos.

Verificación concreta. Las clases de tests referencian Program de la API. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.22 — Configurar WebApplicationFactory y base aislada para tests

Objetivo del paso. Configurar WebApplicationFactory y base aislada para tests. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Crea ApiFactory derivada de WebApplicationFactory; sustituye solo para pruebas el proveedor por EF Core InMemory. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`tests/AceriaApp.Api.Tests/ApiFactory.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Datos;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.AspNetCore.TestHost;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.1.12: HTTP real; base en memoria SOLO en tests automatizados.
public sealed class ApiFactory:WebApplicationFactory<Program>
{
    private readonly string _databaseName="Tests-"+Guid.NewGuid();
    protected override void ConfigureWebHost(IWebHostBuilder builder)
```

```
    {
        builder.UseEnvironment("Development");
        builder.UseSetting("SkipDatabaseInitialization","true");
        builder.ConfigureTestServices(services=>
        {
            services.RemoveAll<DbContextOptions<AceriaDbContext>>();
            services.AddDbContext<AceriaDbContext>(opt=>
                opt.UseInMemoryDatabase(_databaseName));
        });
    }
}
```

Explicación conceptual. La plantilla {id:guid} restringe el parámetro de ruta a GUID válidos. El manejador consulta una orden específica y decide entre Results.Ok y Results.NotFound. Una petición a un GUID inexistente no debería convertirse en 500 ni en una respuesta 200 con contenido vacío. WebApplicationFactory inicia la aplicación en memoria y permite comprobar 200 y 404 sobre los endpoints auténticos.

Verificación concreta. Las pruebas arrancan ASP.NET Core en memoria; la aplicación real conserva LocalDB. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.23 — Probar 200, 404 y DTO mediante GetTests

Objetivo del paso. Probar 200, 404 y DTO mediante GetTests. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Crea GetTests y tres casos: colección 200, GUID inexistente 404, orden insertada recuperada como OrdenDto. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`tests/AceriaApp.Api.Tests/GetTests.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.Extensions.DependencyInjection;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.1.13: dos peticiones HTTP, código y DTO.
public class GetTests
{
    [Fact] public async Task ListaVacia_Es200()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes");
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
    }
    [Fact] public async Task RecursoInexistente_Es404()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes/"+Guid.NewGuid());
        Assert.Equal(HttpStatusCode.NotFound,r.StatusCode);
```

```
    }
    [Fact] public async Task RecursoExistente_ExponeDto()
    {
        using var f=new ApiFactory();
        var id=Guid.NewGuid();
        using(var scope=f.Services.CreateScope())
        {
            var db=scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
            db.Ordenes.Add(new Orden{Id=id,Cliente="ACX-001",Material="CHAPA",Cantidad=10});
            await db.SaveChangesAsync();
        }
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes/"+id);
        var dto=await r.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
        Assert.Equal(id,dto?.Id);
    }
}
```

Explicación conceptual. La plantilla {id:guid} restringe el parámetro de ruta a GUID válidos. El manejador consulta una orden específica y decide entre Results.Ok y Results.NotFound. Una petición a un GUID inexistente no debería convertirse en 500 ni en una respuesta 200 con contenido vacío. WebApplicationFactory inicia la aplicación en memoria y permite comprobar 200 y 404 sobre los endpoints auténticos.

Verificación concreta. dotnet test ejecuta los tres casos con resultado satisfactorio. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

PASO 2.1.24 — Reto: incorporar consulta por cliente y explicar su URI

Objetivo del paso. Reto: incorporar consulta por cliente y explicar su URI. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Como reto, acepta ?cliente=ACX-001 en el GET de colección y aplica Where solo cuando se recibe cliente; crea un test. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.1-diseno-apis-rest` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. La plantilla {id:guid} restringe el parámetro de ruta a GUID válidos. El manejador consulta una orden específica y decide entre Results.Ok y Results.NotFound. Una petición a un GUID inexistente no debería convertirse en 500 ni en una respuesta 200 con contenido vacío. WebApplicationFactory inicia la aplicación en memoria y permite comprobar 200 y 404 sobre los endpoints auténticos.

Verificación concreta. La colección devuelve únicamente órdenes del cliente indicado. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Si algo falla. Si aparece un error de proveedor SQL Server, revisa NuGet, la cadena LocalDB y los servicios registrados; si el código de la respuesta no coincide, inspecciona el patrón MapGet.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`GET /api/v1/ordenes`	200 + JSON

Comportamiento	Resultado esperado
`GET /api/v1/ordenes/{id}`	200 / 404

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.2 — Recursos, verbos, persistencia y respuestas HTTP

Proyecto comprobado: `PROYECTO/2.2-recursos-verbos-codigos`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.2.1 — Recuperar el estado GET funcional sin modificarlo

Objetivo del paso. Recuperar el estado GET funcional sin modificarlo. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Abre la etapa 2.1 ya programada; copia tu solución de trabajo a la carpeta del punto 2.2 y ejecuta los GET existentes. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. El servidor debe recibir solamente Cliente, Material y Cantidad en CrearOrdenRequest. El ID, el Estado y la FechaCreacion pertenecen al control interno del servicio. Por eso no reutilizamos Orden como cuerpo de entrada. Este diseño protege el contrato público y delimita claramente los datos que el cliente puede modificar.

Verificación concreta. La lista sigue devolviendo 200 y el detalle inexistente 404. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.2 — Diseñar la tabla verbo, URI, petición y respuesta

Objetivo del paso. Diseñar la tabla verbo, URI, petición y respuesta. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Construye una tabla con GET/POST/PUT/PATCH/DELETE y las URIs de órdenes; anota cuándo será 201, 204, 400, 404, 409 y 422. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. El servidor debe recibir solamente Cliente, Material y Cantidad en CrearOrdenRequest. El ID, el Estado y la FechaCreacion pertenecen al control interno del servicio. Por eso no reutilizamos Orden como cuerpo de entrada. Este diseño protege el contrato público y delimita claramente los datos que el cliente puede modificar.

Verificación concreta. La tabla coincide con las rutas que programarás, sin acciones Git. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.3 — Separar contrato de entrada y DTO de salida

Objetivo del paso. Separar contrato de entrada y DTO de salida. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En la carpeta Contratos dibuja qué acepta el cliente y qué genera el servidor. No recibas Orden directamente desde JSON. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. El servidor debe recibir solamente Cliente, Material y Cantidad en CrearOrdenRequest. El ID, el Estado y la FechaCreacion pertenecen al control interno del servicio. Por eso no reutilizamos Orden como cuerpo de entrada. Este diseño protege el contrato público y delimita claramente los datos que el cliente puede modificar.

Verificación concreta. Id, Estado y FechaCreacion quedan fuera del contrato de creación. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.4 — Crear CrearOrdenRequest y ActualizarCantidadRequest

Objetivo del paso. Crear CrearOrdenRequest y ActualizarCantidadRequest. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Crea ContratosOrden.cs con CrearOrdenRequest(Cliente, Material, Cantidad) y ActualizarCantidadRequest(Cantidad). Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Contratos/ContratosOrden.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
namespace AceriaApp.Api.Contratos;
// INICIO PASO 2.2.4
public sealed record CrearOrdenRequest(string Cliente,string Material,int Cantidad);
public sealed record ActualizarCantidadRequest(int Cantidad);
// FIN PASO 2.2.4
```

Explicación conceptual. El servidor debe recibir solamente Cliente, Material y Cantidad en CrearOrdenRequest. El ID, el Estado y la FechaCreacion pertenecen al control interno del servicio. Por eso no reutilizamos Orden como cuerpo de entrada. Este diseño protege el contrato público y delimita claramente los datos que el cliente puede modificar.

Verificación concreta. Los dos records compilan y se distinguen del OrdenDto de salida. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.5 — Programar MapPost y recibir JSON de una orden

Objetivo del paso. Programar MapPost y recibir JSON de una orden. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En Program.cs declara MapPost /api/v1/ordenes con CrearOrdenRequest y AceriaDbContext. Explica cómo se enlaza JSON. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    var orden=new
Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")


;
```

Explicación conceptual. El servidor debe recibir solamente Cliente, Material y Cantidad en CrearOrdenRequest. El ID, el Estado y la FechaCreacion pertenecen al control interno del servicio. Por eso no reutilizamos Orden como cuerpo de entrada. Este diseño protege el contrato público y delimita claramente los datos que el cliente puede modificar.

Verificación concreta. La URL acepta POST con Content-Type application/json. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.6 — Persistir la orden con Add y SaveChangesAsync

Objetivo del paso. Persistir la orden con Add y SaveChangesAsync. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Dentro de POST crea una entidad Orden con los campos del contrato, ejecuta db.Ordenes.Add y SaveChangesAsync. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Explicación conceptual. MapPost convierte un cuerpo JSON en CrearOrdenRequest. Al construir Orden y añadirla al DbContext todavía no se ha confirmado una inserción; SaveChangesAsync ejecuta la escritura. CreatedAtRoute utiliza el nombre ObtenerOrdenPorId para incluir un vínculo al recurso y devolver 201 Created. En prácticas se enviará POST y se seguirá Location con GET para demostrar persistencia real.

Verificación concreta. Una nueva fila queda persistida en LocalDB. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.7 — Devolver 201 y Location con CreatedAtRoute

Objetivo del paso. Devolver 201 y Location con CreatedAtRoute. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Devuelve Results.CreatedAtRoute usando el nombre ObtenerOrdenPorId y el identificador generado. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    var orden=new
Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")


;
```

Explicación conceptual. MapPost convierte un cuerpo JSON en CrearOrdenRequest. Al construir Orden y añadirla al DbContext todavía no se ha confirmado una inserción; SaveChangesAsync ejecuta la escritura. CreatedAtRoute utiliza el nombre ObtenerOrdenPorId para incluir un vínculo al recurso y devolver 201 Created. En prácticas se enviará POST y se seguirá Location con GET para demostrar persistencia real.

Verificación concreta. El resultado es 201 con cabecera Location. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.8 — Enviar POST y consultar por GET la orden creada

Objetivo del paso. Enviar POST y consultar por GET la orden creada. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Envía POST con ACX-001, CHAPA-3MM y 10; copia el identificador y ejecuta GET de detalle. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Explicación conceptual. MapPost convierte un cuerpo JSON en CrearOrdenRequest. Al construir Orden y añadirla al DbContext todavía no se ha confirmado una inserción; SaveChangesAsync ejecuta la escritura. CreatedAtRoute utiliza el nombre ObtenerOrdenPorId para incluir un vínculo al recurso y devolver 201 Created. En prácticas se enviará POST y se seguirá Location con GET para demostrar persistencia real.

Verificación concreta. GET muestra los mismos valores guardados por POST. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.9 — Rechazar provisionalmente peticiones sin datos

Objetivo del paso. Rechazar provisionalmente peticiones sin datos. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En 2.2, antes de introducir FluentValidation, añade un BadRequest provisional ante Cliente o Material vacíos y Cantidad no positiva. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. MapPost convierte un cuerpo JSON en CrearOrdenRequest. Al construir Orden y añadirla al DbContext todavía no se ha confirmado una inserción; SaveChangesAsync ejecuta la escritura. CreatedAtRoute utiliza el nombre ObtenerOrdenPorId para incluir un vínculo al recurso y devolver 201 Created. En prácticas se enviará POST y se seguirá Location con GET para demostrar persistencia real.

Verificación concreta. La petición inválida produce 400 y no crea una orden. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.10 — Comprobar la política del taller y respuesta 422

Objetivo del paso. Comprobar la política del taller y respuesta 422. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Rechaza Cantidad superior a 500 por política de capacidad con Results.Problem statusCode 422. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. MapPost convierte un cuerpo JSON en CrearOrdenRequest. Al construir Orden y añadirla al DbContext todavía no se ha confirmado una inserción; SaveChangesAsync ejecuta la escritura. CreatedAtRoute utiliza el nombre ObtenerOrdenPorId para incluir un vínculo al recurso y devolver 201 Created. En prácticas se enviará POST y se seguirá Location con GET para demostrar persistencia real.

Verificación concreta. El JSON de capacidad 501 produce 422; se distingue de 400. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.11 — Programar MapPut con identificador Guid

Objetivo del paso. Programar MapPut con identificador Guid. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Declara MapPut /api/v1/ordenes/{id:guid} con Guid, CrearOrdenRequest y AceriaDbContext. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.2.7: sustitución completa de los campos editables.
app.MapPut("/api/v1/ordenes/{id:guid}",async (Guid id,CrearOrdenRequest request,AceriaDbContext
db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cliente=request.Cliente;orden.Material=request.Material;
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("ReemplazarOrden").WithTags("Ordenes")


;
```

Explicación conceptual. El ejemplo PUT reemplaza los campos editables de una orden ya existente, conservando los valores que controla el servidor. No debe inventar una nueva orden cuando el identificador no existe: en nuestro contrato responde 404. Cuando una orden ya está finalizada la regla del dominio impide editarla y se comunica un conflicto 409.

Verificación concreta. PUT puede buscar un recurso por identificador. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.12 — Modificar Cliente, Material y Cantidad con PUT

Objetivo del paso. Modificar Cliente, Material y Cantidad con PUT. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En el PUT carga FindAsync(id), devuelve 404 si no existe y sustituye los tres campos editables; SaveChangesAsync. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Explicación conceptual. El ejemplo PUT reemplaza los campos editables de una orden ya existente, conservando los valores que controla el servidor. No debe inventar una nueva orden cuando el identificador no existe: en nuestro contrato responde 404. Cuando una orden ya está finalizada la regla del dominio impide editarla y se comunica un conflicto 409.

Verificación concreta. PUT válido responde 204 y mantiene el mismo Guid. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.13 — Comprobar 204 y consultar por GET los cambios

Objetivo del paso. Comprobar 204 y consultar por GET los cambios. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Tras PUT ejecuta GET del mismo recurso y comprueba Cliente, Material y Cantidad nuevos. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Explicación conceptual. El ejemplo PUT reemplaza los campos editables de una orden ya existente, conservando los valores que controla el servidor. No debe inventar una nueva orden cuando el identificador no existe: en nuestro contrato responde 404. Cuando una orden ya está finalizada la regla del dominio impide editarla y se comunica un conflicto 409.

Verificación concreta. La consulta evidencia que el cambio se persistió. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.14 — Programar MapPatch para cambiar solo Cantidad

Objetivo del paso. Programar MapPatch para cambiar solo Cantidad. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Declara MapPatch /api/v1/ordenes/{id:guid}/cantidad con ActualizarCantidadRequest. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.2.8: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
    if(request.Cantidad<=0) return Results.BadRequest(new {error="Cantidad debe ser positiva."});
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")


;
```

Explicación conceptual. El ejemplo PUT reemplaza los campos editables de una orden ya existente, conservando los valores que controla el servidor. No debe inventar una nueva orden cuando el identificador no existe: en nuestro contrato responde 404. Cuando una orden ya está finalizada la regla del dominio impide editarla y se comunica un conflicto 409.

Verificación concreta. PATCH admite un cuerpo que contiene solo Cantidad. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.15 — Comprobar que PATCH no modifica los demás campos

Objetivo del paso. Comprobar que PATCH no modifica los demás campos. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Guarda la nueva cantidad y devuelve Results.Ok(OrdenDto.Desde(orden)); consulta después y verifica que Cliente sigue igual. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Explicación conceptual. El ejemplo PUT reemplaza los campos editables de una orden ya existente, conservando los valores que controla el servidor. No debe inventar una nueva orden cuando el identificador no existe: en nuestro contrato responde 404. Cuando una orden ya está finalizada la regla del dominio impide editarla y se comunica un conflicto 409.

Verificación concreta. Se obtiene 200 y no cambian datos no incluidos en el parche. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.16 — Programar MapDelete para eliminar una orden

Objetivo del paso. Programar MapDelete para eliminar una orden. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Declara MapDelete /api/v1/ordenes/{id:guid}; usa FindAsync, Remove y SaveChangesAsync. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.2.9: eliminar recurso persistido.
app.MapDelete("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede eliminar una orden finalizada.",statusCode:409);
    db.Ordenes.Remove(orden);
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("EliminarOrden").WithTags("Ordenes")

;
```

Explicación conceptual. El endpoint PATCH de este curso cambia exclusivamente Cantidad. La elección de una ruta especializada para ese campo evita introducir JSON Patch en un módulo introductorio. El alumno comprobará que Cliente y Material no cambian después de aplicar la modificación parcial.

Verificación concreta. DELETE existente devuelve 204. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.17 — Verificar 204 seguido de GET 404

Objetivo del paso. Verificar 204 seguido de GET 404. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Llama GET al recurso recién eliminado y después repite DELETE sobre su Id. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Explicación conceptual. El endpoint PATCH de este curso cambia exclusivamente Cantidad. La elección de una ruta especializada para ese campo evita introducir JSON Patch en un módulo introductorio. El alumno comprobará que Cliente y Material no cambian después de aplicar la modificación parcial.

Verificación concreta. El GET devuelve 404 y el recurso sigue ausente. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.18 — Programar la transición de estado finalizar

Objetivo del paso. Programar la transición de estado finalizar. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Declara POST /api/v1/ordenes/{id:guid}/finalizar para establecer Estado=Finalizada y guardar. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Program.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
// INICIO PASO 2.2.10: transición explícita del dominio.
app.MapPost("/api/v1/ordenes/{id:guid}/finalizar",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"La orden ya estaba finalizada.",statusCode:409);
    orden.Estado="Finalizada";
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("FinalizarOrden").WithTags("Ordenes")

;

app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Explicación conceptual. El endpoint PATCH de este curso cambia exclusivamente Cantidad. La elección de una ruta especializada para ese campo evita introducir JSON Patch en un módulo introductorio. El alumno comprobará que Cliente y Material no cambian después de aplicar la modificación parcial.

Verificación concreta. La transición devuelve 200 y el DTO indica Finalizada. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.19 — Bloquear PUT y PATCH para órdenes finalizadas

Objetivo del paso. Bloquear PUT y PATCH para órdenes finalizadas. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En PUT y PATCH comprueba Estado=Finalizada antes de modificar y devuelve Problem con 409. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Explicación conceptual. El endpoint PATCH de este curso cambia exclusivamente Cantidad. La elección de una ruta especializada para ese campo evita introducir JSON Patch en un módulo introductorio. El alumno comprobará que Cliente y Material no cambian después de aplicar la modificación parcial.

Verificación concreta. Ninguna modificación se guarda sobre una orden finalizada. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.20 — Bloquear DELETE y devolver conflicto 409

Objetivo del paso. Bloquear DELETE y devolver conflicto 409. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En DELETE bloquea también la eliminación de una orden finalizada devolviendo 409. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Program.cs`. Busca el método HTTP correspondiente y su llamada a `SaveChangesAsync`. Comprueba el comportamiento indicado en la acción antes de continuar.

Explicación conceptual. El endpoint PATCH de este curso cambia exclusivamente Cantidad. La elección de una ruta especializada para ese campo evita introducir JSON Patch en un módulo introductorio. El alumno comprobará que Cliente y Material no cambian después de aplicar la modificación parcial.

Verificación concreta. Un recurso finalizado permanece en la base tras DELETE fallido. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.21 — Escribir pruebas HTTP POST y Location

Objetivo del paso. Escribir pruebas HTTP POST y Location. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Añade EscrituraTests con POST, verifica 201, Location y GET posterior del recurso creado. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`tests/AceriaApp.Api.Tests/EscrituraTests.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Modelo;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.2.11: probar POST/GET/PUT/PATCH/DELETE con HTTP.
public class EscrituraTests
{
    [Fact] public async Task CrearYConsultar_Es201Y200()
    {
        using var f=new ApiFactory();using var client=f.CreateClient();
        var created=await client.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10));
        Assert.Equal(HttpStatusCode.Created,created.StatusCode);
        Assert.NotNull(created.Headers.Location);
        var get=await client.GetAsync(created.Headers.Location);
        Assert.Equal(HttpStatusCode.OK,get.StatusCode);
    }
    [Fact] public async Task EditarYEliminar_CambiaPersistencia()
    {
        using var f=new ApiFactory();using var c=f.CreateClient();
        var made=await c.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10));
```

```
        var dto=await made.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.NotNull(dto);
        var put=await c.PutAsJsonAsync("/api/v1/ordenes/"+dto.Id,
            new CrearOrdenRequest("ACX2","CHAPA",11));
        Assert.Equal(HttpStatusCode.NoContent,put.StatusCode);
        var patch=await c.PatchAsJsonAsync("/api/v1/ordenes/"+dto.Id+"/cantidad",
            new ActualizarCantidadRequest(12));
        Assert.Equal(HttpStatusCode.OK,patch.StatusCode);
        var changed=await patch.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.Equal(12,changed?.Cantidad);
        var del=await c.DeleteAsync("/api/v1/ordenes/"+dto.Id);
        Assert.Equal(HttpStatusCode.NoContent,del.StatusCode);
        var gone=await c.GetAsync("/api/v1/ordenes/"+dto.Id);
        Assert.Equal(HttpStatusCode.NotFound,gone.StatusCode);
    }
    [Fact] public async Task ReglaCapacidad_Es422()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",501));
        Assert.Equal(HttpStatusCode.UnprocessableEntity,r.StatusCode);
    }
    [Fact] public async Task OrdenFinalizada_NoSeEdita409()
    {
        using var f=new ApiFactory();using var c=f.CreateClient();
        var made=await c.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",2));
        var dto=await made.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.NotNull(dto);
        var done=await c.PostAsync("/api/v1/ordenes/"+dto.Id+"/finalizar",null);
        Assert.Equal(HttpStatusCode.OK,done.StatusCode);
        var put=await c.PutAsJsonAsync("/api/v1/ordenes/"+dto.Id,
            new CrearOrdenRequest("ACX","CHAPA",3));
        Assert.Equal(HttpStatusCode.Conflict,put.StatusCode);
    }
}
```

Explicación conceptual. DELETE busca el recurso y ejecuta Remove y SaveChangesAsync. La primera eliminación devuelve 204; la consulta posterior devuelve 404. La idempotencia describe el estado final del sistema, no la obligación de devolver exactamente el mismo código en peticiones repetidas. Para reproducir 409 se añade la transición POST finalizar y se intenta editar una orden cuyo estado ya no permite modificaciones.

Verificación concreta. El test confirma creación y posterior consulta por HTTP. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.22 — Escribir pruebas HTTP PUT, PATCH y DELETE

Objetivo del paso. Escribir pruebas HTTP PUT, PATCH y DELETE. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En el test del ciclo completo ejecuta PUT, PATCH y DELETE, leyendo el DTO después de cada cambio. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: `tests/AceriaApp.Api.Tests/EscrituraTests.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Modelo;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.2.11: probar POST/GET/PUT/PATCH/DELETE con HTTP.
public class EscrituraTests
{
    [Fact] public async Task CrearYConsultar_Es201Y200()
    {
        using var f=new ApiFactory();using var client=f.CreateClient();
        var created=await client.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10));
        Assert.Equal(HttpStatusCode.Created,created.StatusCode);
        Assert.NotNull(created.Headers.Location);
        var get=await client.GetAsync(created.Headers.Location);
        Assert.Equal(HttpStatusCode.OK,get.StatusCode);
    }
    [Fact] public async Task EditarYEliminar_CambiaPersistencia()
    {
        using var f=new ApiFactory();using var c=f.CreateClient();
        var made=await c.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10));
        var dto=await made.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.NotNull(dto);
        var put=await c.PutAsJsonAsync("/api/v1/ordenes/"+dto.Id,
            new CrearOrdenRequest("ACX2","CHAPA",11));
        Assert.Equal(HttpStatusCode.NoContent,put.StatusCode);
        var patch=await c.PatchAsJsonAsync("/api/v1/ordenes/"+dto.Id+"/cantidad",
            new ActualizarCantidadRequest(12));
        Assert.Equal(HttpStatusCode.OK,patch.StatusCode);
        var changed=await patch.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.Equal(12,changed?.Cantidad);
        var del=await c.DeleteAsync("/api/v1/ordenes/"+dto.Id);
        Assert.Equal(HttpStatusCode.NoContent,del.StatusCode);
        var gone=await c.GetAsync("/api/v1/ordenes/"+dto.Id);
        Assert.Equal(HttpStatusCode.NotFound,gone.StatusCode);
    }
    [Fact] public async Task ReglaCapacidad_Es422()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",501));
        Assert.Equal(HttpStatusCode.UnprocessableEntity,r.StatusCode);
    }
    [Fact] public async Task OrdenFinalizada_NoSeEdita409()
    {
        using var f=new ApiFactory();using var c=f.CreateClient();
        var made=await c.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",2));
        var dto=await made.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.NotNull(dto);
        var done=await c.PostAsync("/api/v1/ordenes/"+dto.Id+"/finalizar",null);
        Assert.Equal(HttpStatusCode.OK,done.StatusCode);
        var put=await c.PutAsJsonAsync("/api/v1/ordenes/"+dto.Id,
            new CrearOrdenRequest("ACX","CHAPA",3));
```

```
        Assert.Equal(HttpStatusCode.Conflict,put.StatusCode);
    }
}
```

Explicación conceptual. DELETE busca el recurso y ejecuta Remove y SaveChangesAsync. La primera eliminación devuelve 204; la consulta posterior devuelve 404. La idempotencia describe el estado final del sistema, no la obligación de devolver exactamente el mismo código en peticiones repetidas. Para reproducir 409 se añade la transición POST finalizar y se intenta editar una orden cuyo estado ya no permite modificaciones.

Verificación concreta. Se comprueban 204, 200, 204 y después 404. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.23 — Probar automáticamente 422 y 409

Objetivo del paso. Probar automáticamente 422 y 409. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Añade pruebas de capacidad 501 y conflicto tras finalizar; ejecuta todo el proyecto xUnit. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. DELETE busca el recurso y ejecuta Remove y SaveChangesAsync. La primera eliminación devuelve 204; la consulta posterior devuelve 404. La idempotencia describe el estado final del sistema, no la obligación de devolver exactamente el mismo código en peticiones repetidas. Para reproducir 409 se añade la transición POST finalizar y se intenta editar una orden cuyo estado ya no permite modificaciones.

Verificación concreta. La batería comprueba las respuestas 422 y 409. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

PASO 2.2.24 — Reto: comprobar idempotencia y efecto persistente

Objetivo del paso. Reto: comprobar idempotencia y efecto persistente. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Como reto envía dos PUT idénticos y dos DELETE sobre el mismo Guid. Distingue efecto final de código HTTP. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.2-recursos-verbos-codigos` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. DELETE busca el recurso y ejecuta Remove y SaveChangesAsync. La primera eliminación devuelve 204; la consulta posterior devuelve 404. La idempotencia describe el estado final del sistema, no la obligación de devolver exactamente el mismo código en peticiones repetidas. Para reproducir 409 se añade la transición POST finalizar y se intenta editar una orden cuyo estado ya no permite modificaciones.

Verificación concreta. El estado del recurso no se duplica y el segundo DELETE no lo restaura. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Si algo falla. Si una escritura devuelve 200 pero no cambia datos, busca SaveChangesAsync; si CreatedAtRoute falla revisa el nombre ObtenerOrdenPorId; si devuelve 404, comprueba el GUID.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`POST /api/v1/ordenes`	201 + Location
`PUT /{id}`	204
`PATCH /{id}/cantidad`	200
`DELETE /{id}`	204 / 404
`POST /{id}/finalizar`	200 / 409
`Cantidad 501`	422

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.3 — Validación efectiva de peticiones con FluentValidation

Proyecto comprobado: `PROYECTO/2.3-validacion-peticiones`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.3.1 — Comprobar endpoints anteriores y localizar sus contratos

Objetivo del paso. Comprobar endpoints anteriores y localizar sus contratos. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Abre el proyecto del punto 2.2 y comprueba GET, POST, PUT, PATCH y DELETE antes de modificarlo. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. ASP.NET Core comprueba aspectos de la deserialización del JSON, FluentValidation aplica reglas semánticas y el manejador aplica decisiones del dominio. Un cuerpo con cantidad 0 falla la semántica; cantidad 501 satisface el rango del contrato pero supera el límite de negocio de 500. El objetivo es demostrar por qué el primer caso produce 400 y el segundo 422.

Verificación concreta. Los tests de escritura siguen pasando. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.2 — Distinguir JSON incorrecto, semántica y negocio

Objetivo del paso. Distinguir JSON incorrecto, semántica y negocio. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Envía tres cuerpos distintos: JSON roto, cantidad 0 y cantidad 501; anota si debe actuar el framework, el validador o el negocio. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. ASP.NET Core comprueba aspectos de la deserialización del JSON, FluentValidation aplica reglas semánticas y el manejador aplica decisiones del dominio. Un cuerpo con cantidad 0 falla la semántica; cantidad 501 satisface el rango del contrato pero supera el límite de negocio de 500. El objetivo es demostrar por qué el primer caso produce 400 y el segundo 422.

Verificación concreta. Distingues sintaxis, semántica y regla de capacidad. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.3 — Instalar FluentValidation.DependencyInjectionExtensions

Objetivo del paso. Instalar FluentValidation.DependencyInjectionExtensions. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Instala FluentValidation.DependencyInjectionExtensions compatible con .NET 8 en el proyecto de API. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. ASP.NET Core comprueba aspectos de la deserialización del JSON, FluentValidation aplica reglas semánticas y el manejador aplica decisiones del dominio. Un cuerpo con cantidad 0 falla la semántica; cantidad 501 satisface el rango del contrato pero supera el límite de negocio de 500. El objetivo es demostrar por qué el primer caso produce 400 y el segundo 422.

Verificación concreta. AbstractValidator y AddValidatorsFromAssemblyContaining se resuelven. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.4 — Crear CrearOrdenValidator y reglas de Cliente

Objetivo del paso. Crear CrearOrdenValidator y reglas de Cliente. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Crea Validadores.cs con CrearOrdenValidator y reglas RuleFor de Cliente con NotEmpty y MaximumLength(100). Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Validacion/Validadores.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Contratos;
using FluentValidation;
namespace AceriaApp.Api.Validacion;
// INICIO PASO 2.3.3
public sealed class CrearOrdenValidator : AbstractValidator<CrearOrdenRequest>
{
    public CrearOrdenValidator()
    {
        RuleFor(x=>x.Cliente).NotEmpty().MaximumLength(100);
        RuleFor(x=>x.Material).NotEmpty().MaximumLength(100);
        RuleFor(x=>x.Cantidad).InclusiveBetween(1,10000);
    }
}
public sealed class ActualizarCantidadValidator : AbstractValidator<ActualizarCantidadRequest>
{
    public ActualizarCantidadValidator() => RuleFor(x=>x.Cantidad).InclusiveBetween(1,10000);
}
// FIN PASO 2.3.3
```

Explicación conceptual. ASP.NET Core comprueba aspectos de la deserialización del JSON, FluentValidation aplica reglas semánticas y el manejador aplica decisiones del dominio. Un cuerpo con cantidad 0 falla la semántica; cantidad 501 satisface el rango del contrato pero supera el límite de negocio de 500. El objetivo es demostrar por qué el primer caso produce 400 y el segundo 422.

Verificación concreta. Un cliente vacío infringe una regla semántica. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.5 — Definir Material y rango válido de Cantidad

Objetivo del paso. Definir Material y rango válido de Cantidad. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En ese validador incorpora Material obligatorio y Cantidad InclusiveBetween(1,10000). Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. ASP.NET Core comprueba aspectos de la deserialización del JSON, FluentValidation aplica reglas semánticas y el manejador aplica decisiones del dominio. Un cuerpo con cantidad 0 falla la semántica; cantidad 501 satisface el rango del contrato pero supera el límite de negocio de 500. El objetivo es demostrar por qué el primer caso produce 400 y el segundo 422.

Verificación concreta. La cantidad 0 o 10001 es inválida; 501 cumple el rango. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.6 — Crear ActualizarCantidadValidator para PATCH

Objetivo del paso. Crear ActualizarCantidadValidator para PATCH. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Añade ActualizarCantidadValidator para el request de PATCH y su regla de cantidad. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Validacion/Validadores.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Contratos;
using FluentValidation;
namespace AceriaApp.Api.Validacion;
// INICIO PASO 2.3.3
public sealed class CrearOrdenValidator : AbstractValidator<CrearOrdenRequest>
{
    public CrearOrdenValidator()
    {
        RuleFor(x=>x.Cliente).NotEmpty().MaximumLength(100);
        RuleFor(x=>x.Material).NotEmpty().MaximumLength(100);
        RuleFor(x=>x.Cantidad).InclusiveBetween(1,10000);
    }
}
public sealed class ActualizarCantidadValidator : AbstractValidator<ActualizarCantidadRequest>
{
    public ActualizarCantidadValidator() => RuleFor(x=>x.Cantidad).InclusiveBetween(1,10000);
}
// FIN PASO 2.3.3
```

Explicación conceptual. CrearOrdenValidator deriva de AbstractValidator y declara reglas NotEmpty, MaximumLength e InclusiveBetween. El validador es código C# ejecutable, no una anotación narrativa. Permite reutilizar reglas en POST y PUT y establecer pruebas de frontera cuando la cantidad es 0, 1, 500, 501 y 10001.

Verificación concreta. PATCH comparte el criterio semántico del campo Cantidad. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.7 — Ejecutar validadores y estudiar ValidationResult

Objetivo del paso. Ejecutar validadores y estudiar ValidationResult. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Instancia los validadores en un test sencillo y observa IsValid y Errors con peticiones válidas e inválidas. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. CrearOrdenValidator deriva de AbstractValidator y declara reglas NotEmpty, MaximumLength e InclusiveBetween. El validador es código C# ejecutable, no una anotación narrativa. Permite reutilizar reglas en POST y PUT y establecer pruebas de frontera cuando la cantidad es 0, 1, 500, 501 y 10001.

Verificación concreta. Comprendes ValidationResult antes de conectar HTTP. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.8 — Crear ValidationFilter genérico de Minimal API

Objetivo del paso. Crear ValidationFilter genérico de Minimal API. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Crea ValidationFilter.cs e implementa IEndpointFilter y su InvokeAsync. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: `src/AceriaApp.Api/Validacion/ValidationFilter.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using FluentValidation;
namespace AceriaApp.Api.Validacion;
// INICIO PASO 2.3.4
public sealed class ValidationFilter<T>(IValidator<T> validator): IEndpointFilter where T:class
{
    public async ValueTask<object?> InvokeAsync(EndpointFilterInvocationContext context,
        EndpointFilterDelegate next)
    {
        var request=context.Arguments.OfType<T>().FirstOrDefault();
        if(request is null) return Results.ValidationProblem(
            new Dictionary<string,string[]> { ["body"]=new[]{"Falta el cuerpo."} });
        var result=await validator.ValidateAsync(request,context.HttpContext.RequestAborted);
        if(!result.IsValid)
        {
            var errors=result.Errors.GroupBy(x=>x.PropertyName)
                .ToDictionary(x=>x.Key,x=>x.Select(e=>e.ErrorMessage).ToArray());
            return Results.ValidationProblem(errors);
        }
        return await next(context);
    }
}
// FIN PASO 2.3.4
```

Explicación conceptual. CrearOrdenValidator deriva de AbstractValidator y declara reglas NotEmpty, MaximumLength e InclusiveBetween. El validador es código C# ejecutable, no una anotación narrativa. Permite reutilizar reglas en POST y PUT y establecer pruebas de frontera cuando la cantidad es 0, 1, 500, 501 y 10001.

Verificación concreta. El filtro recibe EndpointFilterInvocationContext y EndpointFilterDelegate. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.9 — Obtener la petición de EndpointFilterInvocationContext

Objetivo del paso. Obtener la petición de EndpointFilterInvocationContext. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En InvokeAsync usa context.Arguments.OfType().FirstOrDefault para localizar el contrato de la petición. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Explicación conceptual. CrearOrdenValidator deriva de AbstractValidator y declara reglas NotEmpty, MaximumLength e InclusiveBetween. El validador es código C# ejecutable, no una anotación narrativa. Permite reutilizar reglas en POST y PUT y establecer pruebas de frontera cuando la cantidad es 0, 1, 500, 501 y 10001.

Verificación concreta. Un request ausente no provoca NullReferenceException. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.10 — Ejecutar ValidateAsync y respetar RequestAborted

Objetivo del paso. Ejecutar ValidateAsync y respetar RequestAborted. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Invoca ValidateAsync pasando RequestAborted desde HttpContext para respetar cancelaciones. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Explicación conceptual. CrearOrdenValidator deriva de AbstractValidator y declara reglas NotEmpty, MaximumLength e InclusiveBetween. El validador es código C# ejecutable, no una anotación narrativa. Permite reutilizar reglas en POST y PUT y establecer pruebas de frontera cuando la cantidad es 0, 1, 500, 501 y 10001.

Verificación concreta. No continúa una operación que el cliente ha cancelado. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.11 — Agrupar errores por campo con LINQ

Objetivo del paso. Agrupar errores por campo con LINQ. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Agrupa result.Errors por PropertyName y construye un diccionario string[] de mensajes. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Validacion/ValidationFilter.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using FluentValidation;
namespace AceriaApp.Api.Validacion;
// INICIO PASO 2.3.4
public sealed class ValidationFilter<T>(IValidator<T> validator): IEndpointFilter where T:class
{
```

```
    public async ValueTask<object?> InvokeAsync(EndpointFilterInvocationContext context,
        EndpointFilterDelegate next)
    {
        var request=context.Arguments.OfType<T>().FirstOrDefault();
        if(request is null) return Results.ValidationProblem(
            new Dictionary<string,string[]> { ["body"]=new[]{"Falta el cuerpo."} });
        var result=await validator.ValidateAsync(request,context.HttpContext.RequestAborted);
        if(!result.IsValid)
        {
            var errors=result.Errors.GroupBy(x=>x.PropertyName)
                .ToDictionary(x=>x.Key,x=>x.Select(e=>e.ErrorMessage).ToArray());
            return Results.ValidationProblem(errors);
        }
        return await next(context);
    }
}
// FIN PASO 2.3.4
```

Explicación conceptual. ValidationFilter recibe el argumento de la petición, ejecuta ValidateAsync y agrupa errores por propiedad. Si hay fallos devuelve Results.ValidationProblem con el campo errors; solo cuando IsValid es verdadero continúa mediante next(context). Omitir esa condición permitiría procesar datos rechazados.

Verificación concreta. Cada propiedad tiene su colección de errores. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.12 — Responder mediante Results.ValidationProblem

Objetivo del paso. Responder mediante Results.ValidationProblem. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Ante fallo devuelve Results.ValidationProblem(errors) sin invocar el manejador siguiente. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Explicación conceptual. ValidationFilter recibe el argumento de la petición, ejecuta ValidateAsync y agrupa errores por propiedad. Si hay fallos devuelve Results.ValidationProblem con el campo errors; solo cuando IsValid es verdadero continúa mediante next(context). Omitir esa condición permitiría procesar datos rechazados.

Verificación concreta. La respuesta incluye status 400 y el campo errors. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.13 — Permitir next(context) solo cuando IsValid

Objetivo del paso. Permitir next(context) solo cuando IsValid. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Si IsValid es verdadero retorna await next(context); razona qué sucede al omitirlo. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Explicación conceptual. ValidationFilter recibe el argumento de la petición, ejecuta ValidateAsync y agrupa errores por propiedad. Si hay fallos devuelve Results.ValidationProblem con el campo errors; solo cuando IsValid es verdadero continúa mediante next(context). Omitir esa condición permitiría procesar datos rechazados.

Verificación concreta. El endpoint se ejecuta solo cuando la validación pasa. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.14 — Registrar validadores en el contenedor DI

Objetivo del paso. Registrar validadores en el contenedor DI. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Antes de builder.Build registra los validadores mediante AddValidatorsFromAssemblyContaining(). Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: `src/AceriaApp.Api/Program.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using FluentValidation;
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>(); // PASO 2.3.5
```

Explicación conceptual. ValidationFilter recibe el argumento de la petición, ejecuta ValidateAsync y agrupa errores por propiedad. Si hay fallos devuelve Results.ValidationProblem con el campo errors; solo cuando IsValid es verdadero continúa mediante next(context). Omitir esa condición permitiría procesar datos rechazados.

Verificación concreta. El filtro puede resolver IValidator. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.15 — Conectar filtro de CrearOrdenRequest al POST

Objetivo del paso. Conectar filtro de CrearOrdenRequest al POST. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Añade AddEndpointFilter<ValidationFilter>() al MapPost real. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: `src/AceriaApp.Api/Program.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
.WithName("CrearOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()

;
```

Explicación conceptual. ValidationFilter recibe el argumento de la petición, ejecuta ValidateAsync y agrupa errores por propiedad. Si hay fallos devuelve Results.ValidationProblem con el campo errors; solo cuando IsValid es verdadero continúa mediante next(context). Omitir esa condición permitiría procesar datos rechazados.

Verificación concreta. POST con Cliente vacío devuelve 400 en vez de guardar una fila. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.16 — Conectar filtro de CrearOrdenRequest al PUT

Objetivo del paso. Conectar filtro de CrearOrdenRequest al PUT. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Añade el mismo filtro de CrearOrdenRequest al MapPut. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Explicación conceptual. AddValidatorsFromAssemblyContaining registra servicios en el contenedor, pero no aplica reglas de manera automática a un endpoint. Para ejecutar la validación se utiliza AddEndpointFilter en POST, PUT y PATCH con el contrato correspondiente. Los tests deben comprobar el comportamiento HTTP, no limitarse a instanciar un validador.

Verificación concreta. PUT inválido no modifica la orden almacenada. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.17 — Conectar filtro de ActualizarCantidadRequest al PATCH

Objetivo del paso. Conectar filtro de ActualizarCantidadRequest al PATCH. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Añade AddEndpointFilter<ValidationFilter>() al MapPatch. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: `src/AceriaApp.Api/Program.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
.WithName("CambiarCantidad").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()

;
```

Explicación conceptual. AddValidatorsFromAssemblyContaining registra servicios en el contenedor, pero no aplica reglas de manera automática a un endpoint. Para ejecutar la validación se utiliza AddEndpointFilter en POST, PUT y PATCH con el contrato correspondiente. Los tests deben comprobar el comportamiento HTTP, no limitarse a instanciar un validador.

Verificación concreta. PATCH con Cantidad=0 devuelve 400. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.18 — Enviar POST incorrecto y comprobar status 400

Objetivo del paso. Enviar POST incorrecto y comprobar status 400. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Envía un POST con Material vacío y comprueba código, Content-Type y ausencia de inserción. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Explicación conceptual. AddValidatorsFromAssemblyContaining registra servicios en el contenedor, pero no aplica reglas de manera automática a un endpoint. Para ejecutar la validación se utiliza AddEndpointFilter en POST, PUT y PATCH con el contrato correspondiente. Los tests deben comprobar el comportamiento HTTP, no limitarse a instanciar un validador.

Verificación concreta. El código es 400 y la base permanece sin cambios. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.19 — Inspeccionar la propiedad errors del Problem Details

Objetivo del paso. Inspeccionar la propiedad errors del Problem Details. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Examina el JSON Problem Details y localiza en errors el nombre de la propiedad rechazada. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Explicación conceptual. AddValidatorsFromAssemblyContaining registra servicios en el contenedor, pero no aplica reglas de manera automática a un endpoint. Para ejecutar la validación se utiliza AddEndpointFilter en POST, PUT y PATCH con el contrato correspondiente. Los tests deben comprobar el comportamiento HTTP, no limitarse a instanciar un validador.

Verificación concreta. El cliente puede mostrar el mensaje bajo el campo correcto. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.20 — Probar límite 10001 frente a la capacidad 501

Objetivo del paso. Probar límite 10001 frente a la capacidad 501. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Compara un POST de cantidad 10001 con otro de 501 y anota el origen del error. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y el endpoint de `Program.cs` sobre el que se instala el filtro. Comprueba dónde se produce `ValidationProblem` y dónde se llama a `next(context)`.

Explicación conceptual. AddValidatorsFromAssemblyContaining registra servicios en el contenedor, pero no aplica reglas de manera automática a un endpoint. Para ejecutar la validación se utiliza AddEndpointFilter en POST, PUT y PATCH con el contrato correspondiente. Los tests deben comprobar el comportamiento HTTP, no limitarse a instanciar un validador.

Verificación concreta. El primero devuelve 400, el segundo 422. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.21 — Reproducir 409 con orden ya finalizada

Objetivo del paso. Reproducir 409 con orden ya finalizada. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Crea, finaliza e intenta editar una orden: no es un fallo semántico, sino un conflicto de estado. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. Un error de validación 400 debe ofrecer mensajes por propiedad. Una capacidad de taller incumplida se documenta en el contrato como 422. Intentar modificar una orden finalizada genera un 409 porque depende del estado persistido. La diferencia permite a los clientes automatizar el tratamiento de errores.

Verificación concreta. Se recibe 409 y los datos no cambian. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.22 — Crear pruebas de validación mediante HTTP

Objetivo del paso. Crear pruebas de validación mediante HTTP. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Crea ValidacionHttpTests con peticiones reales para Cliente, Material, Cantidad y política de negocio. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`tests/AceriaApp.Api.Tests/ValidacionHttpTests.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using System.Net;
using System.Net.Http.Json;
using System.Text.Json;
using AceriaApp.Api.Contratos;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.3.8: se verifica el filtro ACTIVO, no su mera existencia.
public class ValidacionHttpTests
{
    [Theory]
    [InlineData("","CHAPA",10)]
    [InlineData("ACX","",10)]
    [InlineData("ACX","CHAPA",0)]
    public async Task DatosInvalidos_Es400ConErrors(string cliente,string material,int cantidad)
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest(cliente,material,cantidad));
        Assert.Equal(HttpStatusCode.BadRequest,r.StatusCode);
        using var json=JsonDocument.Parse(await r.Content.ReadAsStringAsync());
        Assert.True(json.RootElement.TryGetProperty("errors",out _));
    }
    [Fact] public async Task CantidadMasDe10000_Es400()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10001));
        Assert.Equal(HttpStatusCode.BadRequest,r.StatusCode);
    }
    [Fact] public async Task ReglaDeNegocio_501_Es422()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",501));
        Assert.Equal(HttpStatusCode.UnprocessableEntity,r.StatusCode);
    }
}
```

Explicación conceptual. Un error de validación 400 debe ofrecer mensajes por propiedad. Una capacidad de taller incumplida se documenta en el contrato como 422. Intentar modificar una orden finalizada genera un 409 porque depende del estado persistido. La diferencia permite a los clientes automatizar el tratamiento de errores.

Verificación concreta. Las respuestas se prueban a través del pipeline completo. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.23 — Verificar que los GET siguen sin regresiones

Objetivo del paso. Verificar que los GET siguen sin regresiones. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Ejecuta los tests GET y de escritura anteriores, además de las nuevas pruebas de validación. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. Un error de validación 400 debe ofrecer mensajes por propiedad. Una capacidad de taller incumplida se documenta en el contrato como 422. Intentar modificar una orden finalizada genera un 409 porque depende del estado persistido. La diferencia permite a los clientes automatizar el tratamiento de errores.

Verificación concreta. El filtrado no rompe las rutas que ya funcionaban. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

PASO 2.3.24 — Reto: personalizar mensajes y comprobar errores

Objetivo del paso. Reto: personalizar mensajes y comprobar errores. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Reto: incorpora WithMessage a una regla y comprueba desde un test HTTP el texto dentro de errors. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.3-validacion-peticiones` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. Un error de validación 400 debe ofrecer mensajes por propiedad. Una capacidad de taller incumplida se documenta en el contrato como 422. Intentar modificar una orden finalizada genera un 409 porque depende del estado persistido. La diferencia permite a los clientes automatizar el tratamiento de errores.

Verificación concreta. El mensaje devuelto es específico y mantiene status 400. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Si algo falla. Si el endpoint acepta datos inválidos, comprueba que se ha añadido AddEndpointFilter al verbo correcto; si aparece una excepción de DI, registra el validador antes de Build.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`Cliente vacío`	400 + errors
`Cantidad 10001`	400 + errors
`Cantidad 501`	422
`Orden finalizada`	409

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.4 — Documento OpenAPI generado y comprobado con Swagger

Proyecto comprobado: `PROYECTO/2.4-documentacion-openapi`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.4.1 — Comprobar el contrato REST que se va a documentar

Objetivo del paso. Comprobar el contrato REST que se va a documentar. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Abre la etapa 2.3 y anota sus rutas, DTOs, estados 200/201/204/400/409/422 antes de documentarla. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.4-documentacion-openapi` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. OpenAPI no es una cadena YAML fabricada para satisfacer un test superficial. Swashbuckle descubre los endpoints realmente declarados en ASP.NET Core y genera el documento JSON. AddEndpointsApiExplorer registra la exploración de rutas y SwaggerDoc aporta metadatos de la versión v1.

Verificación concreta. La futura documentación reflejará endpoints ya comprobados. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.2 — Instalar Swashbuckle.AspNetCore compatible con .NET 8

Objetivo del paso. Instalar Swashbuckle.AspNetCore compatible con .NET 8. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Instala Swashbuckle.AspNetCore 6.6.2 para .NET 8 en el proyecto API, no solo en tests. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.4-documentacion-openapi` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. OpenAPI no es una cadena YAML fabricada para satisfacer un test superficial. Swashbuckle descubre los endpoints realmente declarados en ASP.NET Core y genera el documento JSON. AddEndpointsApiExplorer registra la exploración de rutas y SwaggerDoc aporta metadatos de la versión v1.

Verificación concreta. AddSwaggerGen y UseSwagger están disponibles. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.3 — Crear carpeta Documentacion y extensiones de configuración

Objetivo del paso. Crear carpeta Documentacion y extensiones de configuración. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Crea Documentacion/DocumentacionExtensions.cs y una clase estática DocumentacionExtensions. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Contratos;
using Microsoft.OpenApi.Any;
using Microsoft.OpenApi.Models;
using Swashbuckle.AspNetCore.SwaggerGen;
namespace AceriaApp.Api.Documentacion;
// INICIO PASO 2.4.3
public static class DocumentacionExtensions
{
    public static IServiceCollection AddDocumentacion(this IServiceCollection services)
    {
        services.AddEndpointsApiExplorer();
        services.AddSwaggerGen(options=>
        {
            options.SwaggerDoc("v1",new OpenApiInfo{
                Title="AceriaApp — Órdenes de producción",Version="v1",
                Description="API REST educativa creada con ASP.NET Core 8.",
                Contact=new OpenApiContact{Name="Equipo AceriaApp",Email="equipo@acerialocal"}
            });
            options.SchemaFilter<EjemploCrearOrden>();
        });
        return services;
    }
    public static WebApplication UseDocumentacion(this WebApplication app)
    {
        if(app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI(o=>o.SwaggerEndpoint("/swagger/v1/swagger.json","AceriaApp v1"));
        }
        return app;
    }
}
// FIN PASO 2.4.3
// INICIO PASO 2.4.5
public sealed class EjemploCrearOrden : ISchemaFilter
{
    public void Apply(OpenApiSchema schema,SchemaFilterContext context)
    {
        if(context.Type!=typeof(CrearOrdenRequest)) return;
        schema.Example=new OpenApiObject{
            ["cliente"]=new OpenApiString("ACX-001"),
            ["material"]=new OpenApiString("CHAPA-3MM"),
            ["cantidad"]=new OpenApiInteger(10)
```

```
        };
    }
}
// FIN PASO 2.4.5
```

Explicación conceptual. OpenAPI no es una cadena YAML fabricada para satisfacer un test superficial. Swashbuckle descubre los endpoints realmente declarados en ASP.NET Core y genera el documento JSON. AddEndpointsApiExplorer registra la exploración de rutas y SwaggerDoc aporta metadatos de la versión v1.

Verificación concreta. La configuración OpenAPI queda separada de Program.cs. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.4 — Registrar AddEndpointsApiExplorer y AddSwaggerGen

Objetivo del paso. Registrar AddEndpointsApiExplorer y AddSwaggerGen. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En AddDocumentacion registra AddEndpointsApiExplorer y AddSwaggerGen con opciones. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Contratos;
using Microsoft.OpenApi.Any;
using Microsoft.OpenApi.Models;
using Swashbuckle.AspNetCore.SwaggerGen;
namespace AceriaApp.Api.Documentacion;
// INICIO PASO 2.4.3
public static class DocumentacionExtensions
{
    public static IServiceCollection AddDocumentacion(this IServiceCollection services)
    {
        services.AddEndpointsApiExplorer();
        services.AddSwaggerGen(options=>
        {
            options.SwaggerDoc("v1",new OpenApiInfo{
                Title="AceriaApp — Órdenes de producción",Version="v1",
                Description="API REST educativa creada con ASP.NET Core 8.",
                Contact=new OpenApiContact{Name="Equipo AceriaApp",Email="equipo@acerialocal"}
            });
            options.SchemaFilter<EjemploCrearOrden>();
        });
        return services;
    }
    public static WebApplication UseDocumentacion(this WebApplication app)
    {
        if(app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI(o=>o.SwaggerEndpoint("/swagger/v1/swagger.json","AceriaApp v1"));
        }
```

```
        return app;
    }
}
// FIN PASO 2.4.3
// INICIO PASO 2.4.5
public sealed class EjemploCrearOrden : ISchemaFilter
{
    public void Apply(OpenApiSchema schema,SchemaFilterContext context)
    {
        if(context.Type!=typeof(CrearOrdenRequest)) return;
        schema.Example=new OpenApiObject{
            ["cliente"]=new OpenApiString("ACX-001"),
            ["material"]=new OpenApiString("CHAPA-3MM"),
            ["cantidad"]=new OpenApiInteger(10)
        };
    }
}
// FIN PASO 2.4.5
```

Explicación conceptual. OpenAPI no es una cadena YAML fabricada para satisfacer un test superficial. Swashbuckle descubre los endpoints realmente declarados en ASP.NET Core y genera el documento JSON. AddEndpointsApiExplorer registra la exploración de rutas y SwaggerDoc aporta metadatos de la versión v1.

Verificación concreta. Los servicios descubren los endpoints Minimal API. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.5 — Configurar SwaggerDoc con información y versión v1

Objetivo del paso. Configurar SwaggerDoc con información y versión v1. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Configura SwaggerDoc v1 con título, versión, descripción y contacto de AceriaApp. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Explicación conceptual. OpenAPI no es una cadena YAML fabricada para satisfacer un test superficial. Swashbuckle descubre los endpoints realmente declarados en ASP.NET Core y genera el documento JSON. AddEndpointsApiExplorer registra la exploración de rutas y SwaggerDoc aporta metadatos de la versión v1.

Verificación concreta. La sección info del JSON muestra esos valores. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.6 — Conectar AddDocumentacion en Program.cs

Objetivo del paso. Conectar AddDocumentacion en Program.cs. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En Program.cs llama builder.Services.AddDocumentacion() antes de builder.Build(). Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: `src/AceriaApp.Api/Program.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
builder.Services.AddDocumentacion(); // PASO 2.4.3
```

Explicación conceptual. La aplicación debe ejecutar AddDocumentacion antes de builder.Build y UseDocumentacion después. El primer método registra servicios y el segundo habilita el middleware. Definir una clase de extensión que nunca se invoca en Program.cs no documenta nada.

Verificación concreta. Se registra el generador en el contenedor. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.7 — Configurar UseSwagger y Swagger UI en Development

Objetivo del paso. Configurar UseSwagger y Swagger UI en Development. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En UseDocumentacion coloca UseSwagger y UseSwaggerUI dentro de la condición IsDevelopment. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Explicación conceptual. La aplicación debe ejecutar AddDocumentacion antes de builder.Build y UseDocumentacion después. El primer método registra servicios y el segundo habilita el middleware. Definir una clase de extensión que nunca se invoca en Program.cs no documenta nada.

Verificación concreta. Swagger no se publica automáticamente en Production. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.8 — Conectar UseDocumentacion y abrir /swagger

Objetivo del paso. Conectar UseDocumentacion y abrir /swagger. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Llama app.UseDocumentacion() después de construir la aplicación y abre /swagger desde Visual Studio. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: `src/AceriaApp.Api/Program.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
app.UseDocumentacion(); // PASO 2.4.4
```

Explicación conceptual. La aplicación debe ejecutar AddDocumentacion antes de builder.Build y UseDocumentacion después. El primer método registra servicios y el segundo habilita el middleware. Definir una clase de extensión que nunca se invoca en Program.cs no documenta nada.

Verificación concreta. La interfaz presenta los endpoints del proyecto. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.9 — Descargar el JSON real /swagger/v1/swagger.json

Objetivo del paso. Descargar el JSON real /swagger/v1/swagger.json. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Descarga /swagger/v1/swagger.json como texto JSON, no una cadena YAML fabricada. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Explicación conceptual. La aplicación debe ejecutar AddDocumentacion antes de builder.Build y UseDocumentacion después. El primer método registra servicios y el segundo habilita el middleware. Definir una clase de extensión que nunca se invoca en Program.cs no documenta nada.

Verificación concreta. La petición devuelve 200 y el documento empieza por openapi. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.10 — Identificar info, paths, components y schemas

Objetivo del paso. Identificar info, paths, components y schemas. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Busca las secciones info, paths y components; identifica las rutas de órdenes y sus verbos. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Explicación conceptual. La aplicación debe ejecutar AddDocumentacion antes de builder.Build y UseDocumentacion después. El primer método registra servicios y el segundo habilita el middleware. Definir una clase de extensión que nunca se invoca en Program.cs no documenta nada.

Verificación concreta. GET, POST, PUT, PATCH y DELETE figuran donde corresponde. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.11 — Aplicar WithName y WithTags a los endpoints

Objetivo del paso. Aplicar WithName y WithTags a los endpoints. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En Program.cs examina WithName y WithTags para agrupar los endpoints de órdenes. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Explicación conceptual. WithName, WithSummary, WithTags, Produces, ProducesProblem y ProducesValidationProblem describen operaciones reales. Declarar que un endpoint puede devolver 400 no hace que valide: la validación efectiva ya se incorporó en 2.3. La documentación debe corresponder a lo que el runtime responde.

Verificación concreta. Los nombres se utilizan también en CreatedAtRoute. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.12 — Documentar GET con WithSummary y Produces

Objetivo del paso. Documentar GET con WithSummary y Produces. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Agrega WithSummary y Produces a los GET y comprueba 200 y 404. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: `src/AceriaApp.Api/Program.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
;
```

Explicación conceptual. WithName, WithSummary, WithTags, Produces, ProducesProblem y ProducesValidationProblem describen operaciones reales. Declarar que un endpoint puede devolver 400 no hace que valide: la validación efectiva ya se incorporó en 2.3. La documentación debe corresponder a lo que el runtime responde.

Verificación concreta. El documento describe la lectura y la ausencia del recurso. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.13 — Documentar POST con 201 y ValidationProblem 400

Objetivo del paso. Documentar POST con 201 y ValidationProblem 400. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Documenta POST con Produces(201) y ProducesValidationProblem(400). Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Explicación conceptual. WithName, WithSummary, WithTags, Produces, ProducesProblem y ProducesValidationProblem describen operaciones reales. Declarar que un endpoint puede devolver 400 no hace que valide: la validación efectiva ya se incorporó en 2.3. La documentación debe corresponder a lo que el runtime responde.

Verificación concreta. Swagger enumera creación y error de contrato. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.14 — Documentar PUT y PATCH con códigos alternativos

Objetivo del paso. Documentar PUT y PATCH con códigos alternativos. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Documenta PUT y PATCH con sus códigos, evitando declarar respuestas inventadas. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Explicación conceptual. WithName, WithSummary, WithTags, Produces, ProducesProblem y ProducesValidationProblem describen operaciones reales. Declarar que un endpoint puede devolver 400 no hace que valide: la validación efectiva ya se incorporó en 2.3. La documentación debe corresponder a lo que el runtime responde.

Verificación concreta. La documentación debe coincidir con los Results usados. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.15 — Documentar DELETE y el conflicto 409

Objetivo del paso. Documentar DELETE y el conflicto 409. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Documenta DELETE y el conflicto 409 de órdenes finalizadas. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Explicación conceptual. WithName, WithSummary, WithTags, Produces, ProducesProblem y ProducesValidationProblem describen operaciones reales. Declarar que un endpoint puede devolver 400 no hace que valide: la validación efectiva ya se incorporó en 2.3. La documentación debe corresponder a lo que el runtime responde.

Verificación concreta. El consumidor puede anticipar una eliminación rechazada. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.16 — Crear SchemaFilter EjemploCrearOrden

Objetivo del paso. Crear SchemaFilter EjemploCrearOrden. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Crea EjemploCrearOrden : ISchemaFilter y asigna OpenApiObject para los tres campos del request. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: `src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using AceriaApp.Api.Contratos;
using Microsoft.OpenApi.Any;
using Microsoft.OpenApi.Models;
using Swashbuckle.AspNetCore.SwaggerGen;
namespace AceriaApp.Api.Documentacion;
// INICIO PASO 2.4.3
public static class DocumentacionExtensions
{
    public static IServiceCollection AddDocumentacion(this IServiceCollection services)
    {
        services.AddEndpointsApiExplorer();
        services.AddSwaggerGen(options=>
        {
            options.SwaggerDoc("v1",new OpenApiInfo{
                Title="AceriaApp — Órdenes de producción",Version="v1",
                Description="API REST educativa creada con ASP.NET Core 8.",
                Contact=new OpenApiContact{Name="Equipo AceriaApp",Email="equipo@acerialocal"}
            });
            options.SchemaFilter<EjemploCrearOrden>();
        });
        return services;
    }
    public static WebApplication UseDocumentacion(this WebApplication app)
    {
        if(app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI(o=>o.SwaggerEndpoint("/swagger/v1/swagger.json","AceriaApp v1"));
        }
        return app;
    }
}
// FIN PASO 2.4.3
// INICIO PASO 2.4.5
public sealed class EjemploCrearOrden : ISchemaFilter
{
    public void Apply(OpenApiSchema schema,SchemaFilterContext context)
    {
        if(context.Type!=typeof(CrearOrdenRequest)) return;
        schema.Example=new OpenApiObject{
```

```
            ["cliente"]=new OpenApiString("ACX-001"),
            ["material"]=new OpenApiString("CHAPA-3MM"),
            ["cantidad"]=new OpenApiInteger(10)
        };
    }
}
// FIN PASO 2.4.5
```

Explicación conceptual. SchemaFilter permite mostrar en components.schemas un ejemplo de CrearOrdenRequest con cliente, material y cantidad. El ejemplo no crea una fila en SQL Server; explica el cuerpo que puede enviar el consumidor. El DTO de salida, en cambio, describe la representación de una orden ya existente.

Verificación concreta. El esquema contiene cliente, material y cantidad ilustrativos. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.17 — Inspeccionar example de CrearOrdenRequest en Swagger

Objetivo del paso. Inspeccionar example de CrearOrdenRequest en Swagger. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En JSON comprueba components.schemas.CrearOrdenRequest.example y explica que no son datos persistidos. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Explicación conceptual. SchemaFilter permite mostrar en components.schemas un ejemplo de CrearOrdenRequest con cliente, material y cantidad. El ejemplo no crea una fila en SQL Server; explica el cuerpo que puede enviar el consumidor. El DTO de salida, en cambio, describe la representación de una orden ya existente.

Verificación concreta. El ejemplo aparece en la sección reusable de esquemas. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.18 — Ejecutar un POST desde Swagger UI

Objetivo del paso. Ejecutar un POST desde Swagger UI. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En Swagger UI utiliza Try it out para enviar POST de orden y observar 201 y Location. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Explicación conceptual. SchemaFilter permite mostrar en components.schemas un ejemplo de CrearOrdenRequest con cliente, material y cantidad. El ejemplo no crea una fila en SQL Server; explica el cuerpo que puede enviar el consumidor. El DTO de salida, en cambio, describe la representación de una orden ya existente.

Verificación concreta. Se crea un recurso de verdad desde el explorador HTTP. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.19 — Consultar la orden creada desde la propia UI

Objetivo del paso. Consultar la orden creada desde la propia UI. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Copia el GUID obtenido y ejecuta GET de detalle desde la UI. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Explicación conceptual. SchemaFilter permite mostrar en components.schemas un ejemplo de CrearOrdenRequest con cliente, material y cantidad. El ejemplo no crea una fila en SQL Server; explica el cuerpo que puede enviar el consumidor. El DTO de salida, en cambio, describe la representación de una orden ya existente.

Verificación concreta. GET responde 200 con el DTO de la misma orden. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.20 — Crear tests HTTP del documento OpenAPI real

Objetivo del paso. Crear tests HTTP del documento OpenAPI real. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. En ContratoOpenApiTests, usa WebApplicationFactory para solicitar el JSON real. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código real contrastado: **`tests/AceriaApp.Api.Tests/ContratoOpenApiTests.cs`**. Es el estado final del archivo en la solución verificada; si ya has creado parte del mismo en pasos anteriores, incorpora solo lo que corresponde a esta actividad y conserva lo demás.

```
using System.Net;
using System.Text.Json;
using Microsoft.AspNetCore.Hosting;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.4.8: abrir OpenAPI JSON REAL mediante HTTP.
public class ContratoOpenApiTests
{
    [Fact] public async Task OpenApi_ContieneRutasYVersion()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/swagger/v1/swagger.json");
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
        using var json=JsonDocument.Parse(await r.Content.ReadAsStringAsync());
```

```
Assert.Equal("v1",json.RootElement.GetProperty("info").GetProperty("version").GetString());
        var paths=json.RootElement.GetProperty("paths");
        Assert.True(paths.TryGetProperty("/api/v1/ordenes",out var resource));
        Assert.True(resource.TryGetProperty("get",out _));
        Assert.True(resource.TryGetProperty("post",out _));
    }
    [Fact] public async Task OpenApi_IncluyeEjemplo()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/swagger/v1/swagger.json");
        using var json=JsonDocument.Parse(await r.Content.ReadAsStringAsync());
        var schemas=json.RootElement.GetProperty("components").GetProperty("schemas");
        Assert.True(schemas.TryGetProperty("CrearOrdenRequest",out var schema));
        Assert.True(schema.TryGetProperty("example",out _));
    }
    [Fact] public async Task Produccion_NoExponeSwagger()
    {
        using var f=new ApiFactory();
        using var other=f.WithWebHostBuilder(b=>b.UseEnvironment("Production"));
        var r=await other.CreateClient().GetAsync("/swagger/v1/swagger.json");
        Assert.Equal(HttpStatusCode.NotFound,r.StatusCode);
    }
}
```

Explicación conceptual. SchemaFilter permite mostrar en components.schemas un ejemplo de CrearOrdenRequest con cliente, material y cantidad. El ejemplo no crea una fila en SQL Server; explica el cuerpo que puede enviar el consumidor. El DTO de salida, en cambio, describe la representación de una orden ya existente.

Verificación concreta. El test descarga la respuesta del servidor y la analiza. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.21 — Comprobar rutas GET y POST en paths

Objetivo del paso. Comprobar rutas GET y POST en paths. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Comprueba info.version y rutas de paths con operaciones get y post. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Explicación conceptual. Swagger UI facilita explorar las operaciones y ejecutarlas desde el navegador. WebApplicationFactory descarga /swagger/v1/swagger.json y contrasta versión, paths y schemas. Otra prueba establece Production y exige que el endpoint de documentación no se publique. Así se comprueba el documento generado y no una cadena aislada del servidor.

Verificación concreta. Un cambio de ruta erróneo hace fallar el contrato. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.22 — Comprobar schema y ejemplo de petición

Objetivo del paso. Comprobar schema y ejemplo de petición. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Agrega un test que compruebe CrearOrdenRequest y su propiedad example en components.schemas. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Explicación conceptual. Swagger UI facilita explorar las operaciones y ejecutarlas desde el navegador. WebApplicationFactory descarga /swagger/v1/swagger.json y contrasta versión, paths y schemas. Otra prueba establece Production y exige que el endpoint de documentación no se publique. Así se comprueba el documento generado y no una cadena aislada del servidor.

Verificación concreta. El esquema se genera a partir del contrato público. Usa la herramienta HTTP o el depurador solo si este paso interactúa con una ruta; los cambios de configuración y modelo se verifican primero con la compilación.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.23 — Comprobar que Production responde 404 para Swagger

Objetivo del paso. Comprobar que Production responde 404 para Swagger. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Ejecuta el test en Environment Production y solicita /swagger/v1/swagger.json. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Referencia de código: `DocumentacionExtensions.cs`, `Program.cs` o `ContratoOpenApiTests.cs` según el caso. La prueba debe leer el documento generado, no un texto YAML escrito a mano.

Explicación conceptual. Swagger UI facilita explorar las operaciones y ejecutarlas desde el navegador. WebApplicationFactory descarga /swagger/v1/swagger.json y contrasta versión, paths y schemas. Otra prueba establece Production y exige que el endpoint de documentación no se publique. Así se comprueba el documento generado y no una cadena aislada del servidor.

Verificación concreta. El endpoint no se publica y responde 404. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

PASO 2.4.24 — Reto: documentar PATCH y verificar su summary en JSON

Objetivo del paso. Reto: documentar PATCH y verificar su summary en JSON. Al terminar esta actividad habrás realizado una modificación de código o comprobado una propiedad concreta del contrato REST.

Acciones guiadas. Reto: añade un resumen específico a PATCH y comprueba que paths muestra summary actualizado. Trabaja sobre la copia de tu solución de esta etapa y guarda el archivo antes de ejecutar la aplicación. Cuando el código usa `MapGet`, `MapPost`, `MapPut`, `MapPatch` o `MapDelete`, consulta también la tabla de verbos del punto para justificar la URI y la respuesta.

Código relacionado: consulta los archivos del proyecto `PROYECTO/2.4-documentacion-openapi` para contrastar las declaraciones que acabas de añadir, sin copiar anticipadamente etapas posteriores.

Explicación conceptual. Swagger UI facilita explorar las operaciones y ejecutarlas desde el navegador. WebApplicationFactory descarga /swagger/v1/swagger.json y contrasta versión, paths y schemas. Otra prueba establece Production y exige que el endpoint de documentación no se publique. Así se comprueba el documento generado y no una cadena aislada del servidor.

Verificación concreta. Se verifica que el documento evoluciona al modificar el código. Ejecuta además las pruebas xUnit del proyecto de esta etapa para confirmar que el ejercicio no ha roto los endpoints anteriores.

Si algo falla. Si Swagger JSON devuelve 404 en Development revisa UseDocumentacion y la variable de entorno; si falta una operación, comprueba sus metadatos y el registro en Program.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`GET /swagger/v1/swagger.json`	200 en Development
`info.version`	v1
`paths contiene GET y POST`	Rutas descubiertas
`schemas contiene CrearOrdenRequest`	Contrato registrado
`GET /swagger/v1/swagger.json`	404 en Production

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.
